**Universidad Mariano Gálvez de Guatemala**
**Boca del Monte**

Ingeniería en Sistemas. Ciclo VIII
Jornada Sábado.
**ARQUITECTURA DE COMPUTADORAS II**
EDGAR RENÉ ORNELIS HOIL




Kevin Raí Salazar Pérez | 690-15-3698
Wuili Gustavo Sumale Pacheco | 7691-22-11007
Luis Fernando Lima Ixcuná | 7690-20-17409

## Reporte técnico del proyecto

Diagnóstico y monitorización de hardware mediante ensamblador x86-64

## Introducción

LowMonitor es una herramienta de diagnóstico de hardware desarrollada en lenguaje ensamblador x86-64 para sistemas Linux. Su funcionamiento se apoya en instrucciones del procesador y llamadas directas al sistema operativo, sin utilizar bibliotecas externas de alto nivel. El proyecto permite examinar la identificación del CPU, los puertos de entrada y salida, la integridad de memoria, el rendimiento del procesador, la jerarquía de memoria y las operaciones vectoriales SIMD.

El código está organizado en seis módulos independientes, coordinados por un punto de entrada común y respaldados por rutinas auxiliares. El ensamblado se efectúa con NASM y el enlace con GNU ld, produciendo un ejecutable ELF de 64 bits. La ejecución principal está diseñada para un usuario regular; determinadas operaciones de E/S requieren privilegios adicionales para acceder efectivamente al hardware.

## Arquitectura y tecnologías

La solución emplea NASM 2.15 o superior, GNU binutils (ld) y Make para automatizar el proceso de construcción. main.asm implementa el punto de entrada _start y coordina la ejecución secuencial de los seis módulos. Cada archivo publica sus rutinas mediante global y reutiliza procedimientos de utils.asm mediante extern.

Las utilidades print_string, print_uint y print_fixed2 resuelven la salida por consola y la conversión numérica. En particular, print_uint prepara la representación decimal completa antes de llamar a write: la instrucción syscall modifica RCX y R11, por lo que realizarla durante el bucle de conversión comprometería el contador de iteraciones. Las operaciones del sistema se invocan mediante la convención Linux x86-64, con el número de syscall en RAX.

Los componentes fuente contemplados son main.asm, utils.asm, modulo1_cpu.asm, modulo2_ioports.asm, modulo3_memoria.asm, modulo4_rendimiento.asm, modulo5_jerarquia.asm y modulo6_simd.asm. La construcción genera archivos objeto ELF64 y los enlaza en el binario lowmonitor.

## Descripción de los módulos

### Identificación del procesador

Se utiliza CPUID para consultar el nombre comercial y las capacidades del procesador. Las hojas 0x80000002 a 0x80000004 proporcionan la cadena de identificación, mientras que las hojas 1 y 7 permiten reconocer extensiones SSE, SSE2, AVX, AVX2 y, cuando corresponda, AVX-512. La evaluación de cada bit se realiza mediante BT y saltos condicionales.

En las pruebas realizadas sobre Debian x86-64, el programa identificó un Intel Core Processor (Skylake) y las extensiones SSE, SSE2, AVX y AVX2. Los datos coincidieron con los obtenidos mediante lscpu.

### Monitorización de puertos de entrada y salida

El segundo módulo demuestra instrucciones IN y OUT de 8, 16 y 32 bits, utilizando DX para direccionar los puertos. Se contemplan puertos estándar, incluido el puerto de diagnóstico POST 0x80. Estas instrucciones están sujetas a las restricciones de protección del procesador y, en Linux, requieren el privilegio CAP_SYS_RAWIO para completar accesos directos.

Para permitir la ejecución sin privilegios, se registra un manejador de SIGSEGV mediante rt_sigaction. Cuando una instrucción privilegiada provoca una excepción de protección general (vector 13), el manejador identifica el contexto de la excepción, registra el acceso denegado y recupera la ejecución en la siguiente operación; al finalizar se restaura el tratamiento habitual de la señal. Se distinguen los estados [OK], [--] para lecturas 0xFF y [DENEGADO] cuando el acceso no está autorizado.

La opción make caps permite asignar CAP_SYS_RAWIO al ejecutable para realizar pruebas autorizadas de puertos, mientras que make nocaps revierte dicha capacidad. Esta configuración debe utilizarse únicamente en entornos controlados y con permisos adecuados.

## Diagnóstico de integridad de memoria

La comprobación se ejecuta sobre un búfer de 1 MiB mediante tres patrones: 0xAA, 0x55 y una secuencia incremental. Después de escribir cada patrón, el módulo compara el contenido esperado contra el leído. Las diferencias se calculan con XOR y los bits distintos se contabilizan con POPCNT. La memoria total del sistema se obtiene a través de sysinfo.

Los tres patrones de prueba finalizaron sin errores y se detectaron 7943 MB de memoria total. Una prueba de inyección, que modificó un byte de 0xAA a 0x00, permitió detectar los cuatro bits alterados previstos, confirmando el funcionamiento del conteo.

## Evaluación del rendimiento del CPU

El módulo estima la frecuencia del contador de tiempo mediante RDTSC, con serialización mediante CPUID, y utiliza nanosleep para establecer un intervalo de medición de aproximadamente 100 ms. La carga de referencia ejecuta 100 millones de iteraciones con cinco instrucciones por iteración, equivalente a 500 millones de instrucciones conocidas.

En las pruebas sobre Debian se registraron aproximadamente 172,3 millones de ciclos, un CPI de 0,34, un IPC de 2,90 y una frecuencia del TSC cercana a 1,00 GHz. Estos indicadores se calcularon a partir de la cantidad conocida de instrucciones del benchmark y no equivalen a mediciones de instrucciones retiradas mediante contadores PMU.

## Análisis de la jerarquía de memoria

Para estimar las latencias se aplica una técnica de seguimiento dependiente de punteros (pointer chasing) con recorridos permutados. Las muestras utilizan búferes representativos de L1 (16 KB), L2 (256 KB), L3 (8 MB) y RAM (64 MB), con nodos distribuidos en líneas de caché de 64 bytes. El recorrido evita el acceso secuencial, cuya predicción por hardware puede subestimar las latencias.

La permutación sigue idx = (k × PASO) mod N, con N potencia de dos y PASO impar, lo que permite recorrer el conjunto de nodos en orden disperso. En las pruebas sobre Debian se obtuvieron 1,00 ciclos por acceso a registros; 3,04 en L1; 7,52 en L2; 10,84 en L3 y 175,25 en RAM. La tendencia resulta coherente con los niveles de memoria observados.

## Procesamiento SIMD con AVX2

El módulo adicional verifica el soporte AVX2 con CPUID y utiliza registros YMM de 256 bits para procesar ocho valores de precisión simple en paralelo. Se ejecutan suma vectorizada (vaddps), multiplicación (vmulps) y comparación (vcmpps); vzeroupper reduce penalizaciones al alternar código AVX y SSE. Un benchmark basado en RDTSC estima el rendimiento en GFLOPS.

Las operaciones de suma, multiplicación y comparación vectorial se ejecutaron correctamente. En el entorno virtualizado de Debian se midieron 0,14 GFLOPS. La virtualización constituye un posible factor de degradación del rendimiento, aunque para determinar su efecto preciso serían necesarias pruebas adicionales sobre hardware físico.

## Entornos de prueba y resultados

Las pruebas se realizaron en dos entornos distintos. El primero utilizó Debian x86-64, un procesador Skylake, cuatro CPUs visibles y aproximadamente 8 GB de RAM. El segundo utilizó Ubuntu 26.04.1, kernel 7.0.0-30-generic, arquitectura x86_64 y un procesador Intel Core i5-7200U. Esta distinción permite interpretar correctamente las diferencias de rendimiento y capacidad.

En Ubuntu se detectaron 11310 MB de memoria y una frecuencia TSC aproximada de 2,71 GHz, con CPI de 0,40 e IPC de 2,49. Las latencias L1, L2, L3 y RAM fueron de 3,99, 20,94, 260,26 y 316,83 ciclos por acceso, respectivamente, y el rendimiento SIMD alcanzó 15,38 GFLOPS. Las variaciones observadas son compatibles con las diferencias de hardware y condiciones de ejecución.

La validación cruzada se realizó mediante lscpu, free, /proc/meminfo, /proc/cpuinfo y /sys. El identificador del procesador y las banderas de instrucciones coinciden con las fuentes consultadas; la memoria reportada fue de 7943 MB en las tres lecturas; la frecuencia resultó coherente en el orden de 1 GHz; y la latencia aumentó al avanzar por los niveles de memoria. Los intentos de E/S no autorizada son compatibles con la protección del sistema operativo.

| Indicador | Debian | Ubuntu |
|---|---|---|
| CPU / arquitectura | Skylake, x86-64 | Intel Core i5-7200U, x86-64 |

| | | |
|---|---|---|
| Memoria total | 7943 MB | 11310 MB |
| TSC / CPI / IPC | 1,00 GHz / 0,34 / 2,90 | 2,71 GHz / 0,40 / 2,49 |
| Latencia L1 / RAM | 3,04 / 175,25 ciclos | 3,99 / 316,83 ciclos |
| AVX2 / rendimiento | Operativo / 0,14 GFLOPS | Operativo / 15,38 GFLOPS |

### Construcción y ejecución

La compilación requiere NASM, binutils y make. Desde la carpeta del proyecto, las órdenes se ejecutan en el orden siguiente:

```
sudo apt install nasm binutils make
make
./lowmonitor
```

La orden make ensambla los módulos, genera los archivos objeto y produce el ejecutable ELF64. Para pruebas autorizadas con capacidades de E/S, se dispone de la secuencia opcional siguiente:

```
make caps
./lowmonitor
make nocaps
```

## Evidencias documentales

Las siguientes figuras presentan el entorno de ejecución, el proceso de compilación, la arquitectura de enlace y los resultados obtenidos durante las pruebas del programa.




*Figura 1. Identificación del sistema operativo y la arquitectura del entorno de prueba.*

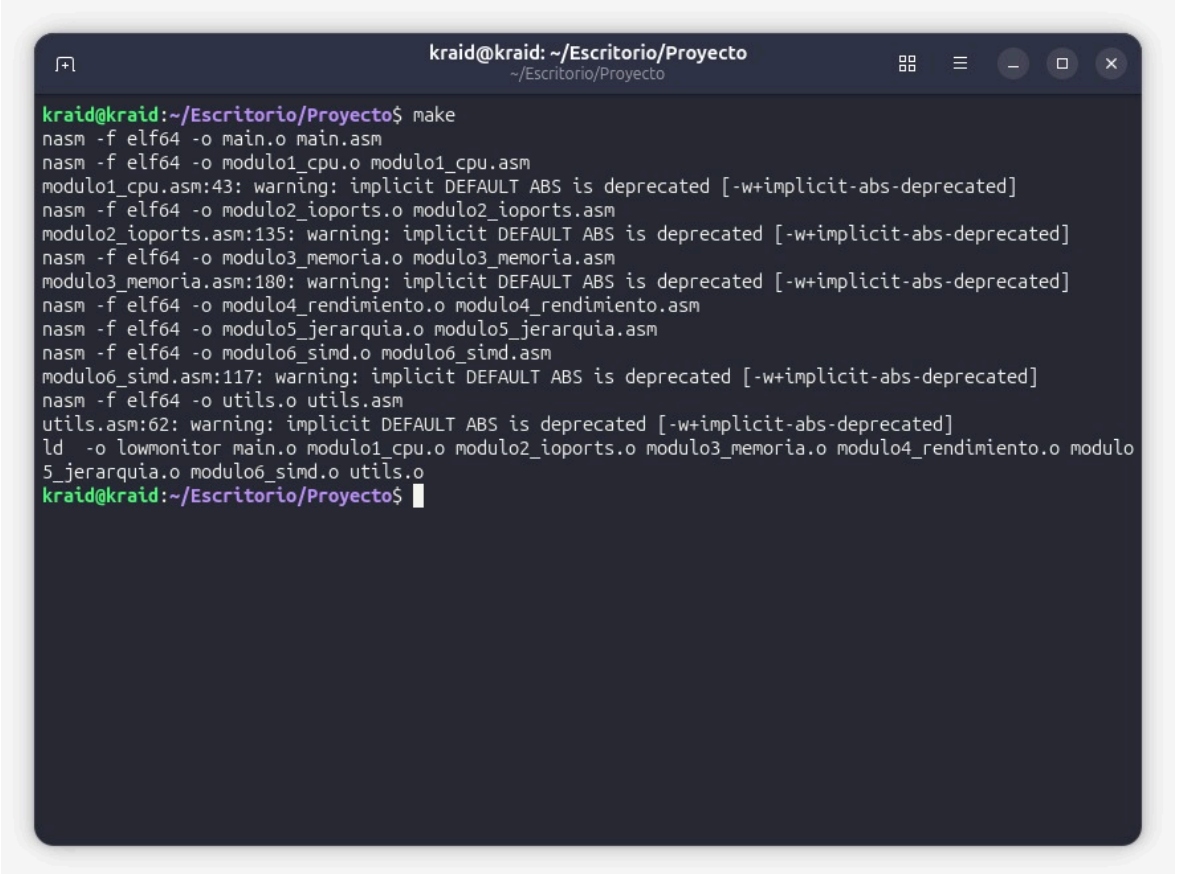



*Figura 2. Salida de la compilación de los módulos con NASM y GNU ld.0*

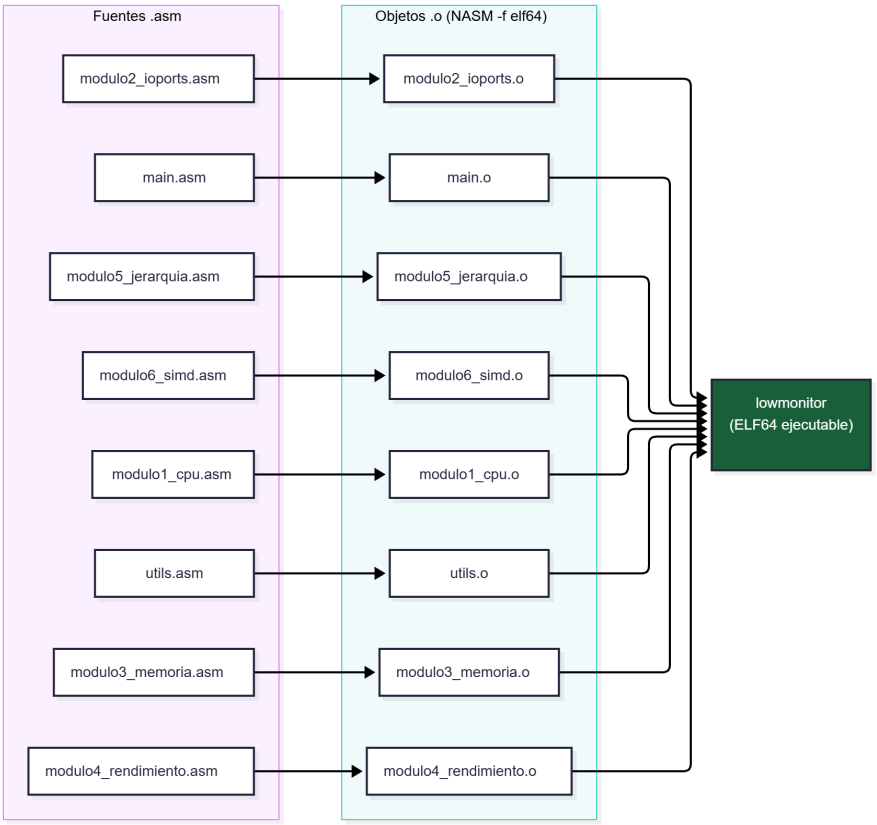



*Figura 3. Flujo de ensamblado, generación de objetos y enlace del ejecutable ELF64.*

```
kraid@kraid: ~/Escritorio/P
~/Escritorio/Proyecto

kraid@kraid:~/Escritorio/Proyecto$ ./lowmonitor
--- Modulo 1: Identificacion del CPU ---
Procesador: Intel(R) Core(TM) i5-7200U CPU @ 2.50GHz
Arquitectura: x86_64
Extensiones: SSE SSE2 AVX AVX2

--- Modulo 2: Monitor de puertos E/S ---
Modo: usuario regular (ioperm denegado, errno 1)
  IN/OUT se ejecutan igual; si el CPU los bloquea, la excepcion
  se captura y se reporta (vector 13 = #GP, proteccion general).
Puerto 60h (teclado): [DENEGADO] excepcion vector 13
Puerto 61h (panel):   [DENEGADO] excepcion vector 13
Puerto 3F8h (COM1):   [DENEGADO] excepcion vector 13
Anchos de E/S demostrados (direccionamiento por DX):
  IN  8  bits (in al, dx)  puerto 60h  [DENEGADO] excepcion vector 13
  IN  16 bits (in ax, dx)  puerto 3F8h [DENEGADO] excepcion vector 13
  IN  32 bits (in eax, dx) puerto 3F8h [DENEGADO] excepcion vector 13
  OUT 8  bits (out dx, al)  puerto 80h [DENEGADO] excepcion vector 13
  OUT 16 bits (out dx, ax)  puerto 80h [DENEGADO] excepcion vector 13
  OUT 32 bits (out dx, eax) puerto 80h [DENEGADO] excepcion vector 13

--- Modulo 3: Diagnostico de memoria ---
Patron 0xAA:              PASADA (0 errores)
Patron 0x55:              PASADA (0 errores)
Patron incremental:       PASADA (0 errores)
Memoria total:            11310 MB detectada
Buffer probado:           1024 KB
```

*Figura 4. Resultados de los módulos de identificación, E/S y memoria.*

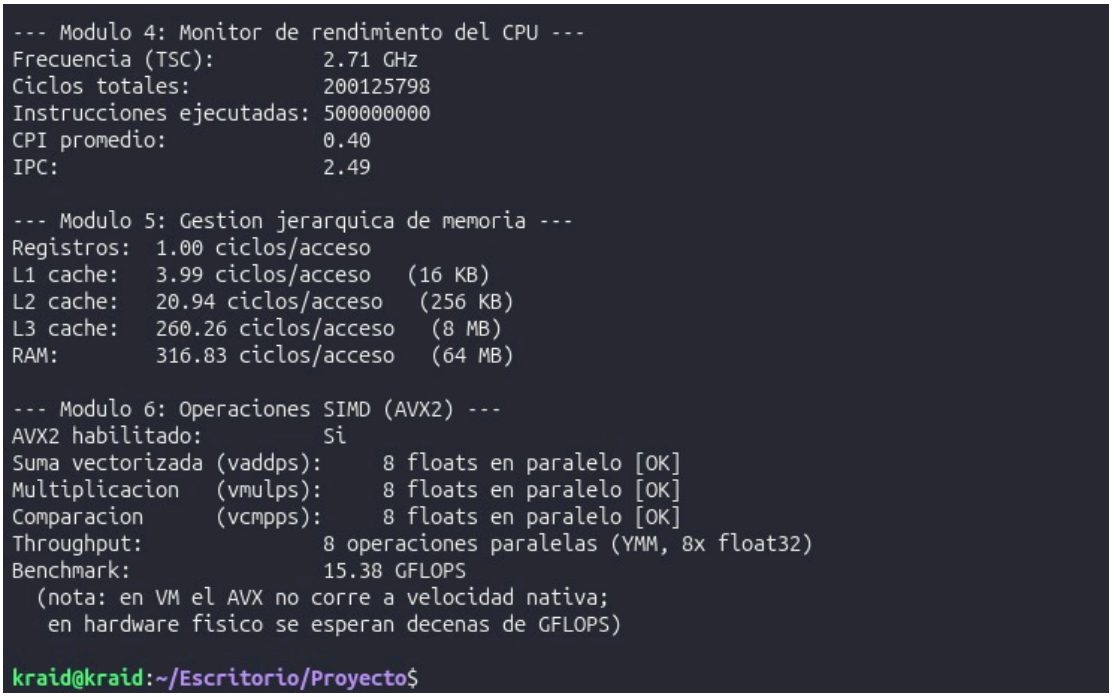



*Figura 5. Resultados de rendimiento, latencias de memoria y operaciones SIMD.*

## Conclusiones

LowMonitor demuestra la aplicación práctica de instrucciones x86-64, acceso controlado a puertos, llamadas directas al kernel, medición basada en ciclos y operaciones vectoriales AVX2. La modularidad del proyecto facilita la validación independiente de cada función y su integración en un ejecutable común.

Los resultados de integridad de memoria y de identificación del procesador cuentan con verificaciones frente a herramientas nativas de Linux. Las mediciones de rendimiento y memoria deben interpretarse dentro de las limitaciones de su metodología y del ambiente de ejecución, especialmente cuando se comparan máquinas virtualizadas con hardware distinto.

## Referencia del proyecto

Repositorio del código fuente: https://github.com/krsalazar/lowmonitor